

☆ SYSTEM DESIGN HANDBOOK ☆

1/10

1. SYSTEM DESIGN FUNDAMENTALS

What is System Design?

System Design is the process of designing **scalable, reliable, and efficient** software systems to meet specific requirements.

Why do companies ask System Design?

- To evaluate how you break down a problem.
- To check your understanding of scale.
- To assess trade-off decisions.
- To see how you design real world systems.

Functional Requirements (What the system should do)

- User should be able to register/login.
- User should be able to upload data.
- User should be able to search data.
- ...

Non-Functional Requirements (How well the system should do it)

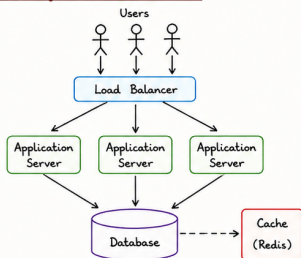
- Scalability
- Reliability
- Availability
- Maintainability
- Performance
- Security
- ...

These define the quality attributes of a system.
Always clarify NFRs in the interview! ☆

Key Quality Attributes

- **Scalability** - Handle growing number of users
- **Reliability** - System works correctly even in failures
- **Availability** - System is up and running most of the time
- **Maintainability** - Easy to update, fix and extend
- **Performance** - Low latency, high throughput
- **Security** - Protect data and ensure privacy

Basic High Level Architecture



Interview Tip

Always start by understanding the problem and gathering requirements.
Ask the right questions before jumping into the design.



☆ SYSTEM DESIGN HANDBOOK ☆

2/10

2. LOAD BALANCER

What is Load Balancer?

A Load Balancer distributes incoming network traffic across multiple servers to ensure no single server is overwhelmed.

Why do we need Load Balancer?

- Distribute traffic evenly across servers.
- Prevent server overload.
- Improve system availability and reliability.
- Enable horizontal scaling.
- Perform health checks.

Types of Load Balancer

- **Layer 4 Load Balancer (Transport Layer)**
 - Works on IP and Port.
 - Uses protocols like TCP, UDP.
 - Faster and simple.
- **Layer 7 Load Balancer (Application Layer)**
 - Works on HTTP/HTTPS.
 - Routes based on URL, headers, cookies, etc.
 - Smarter and more feature rich.

Common Algorithms

- **Round Robin** - Distributes requests in rotation.
- **Least Connections** - Sends request to server with least active connections.
- **IP Hash** - Routes based on client IP.
- **Weighted Round Robin** - Considers server weights.
- **Least Response Time** - Chooses server with lowest response time.

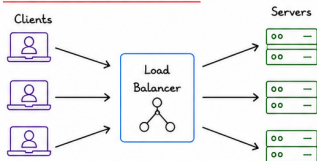


Interview Tip

Always mention the type of load balancer, algorithm, health checks, and failover strategy in your design.



How Load Balancer Works?



Key Features

- **Health Checks** - Regularly checks server health.
- **Failover** - Removes unhealthy servers from rotation.
- **SSL Termination** - Handles SSL encryption.
- **Sticky Sessions** - Ensures same client goes to same server.
- **DDoS Protection** - Helps absorb and filter attacks.

Health Check Example



Popular Load Balancers

- Nginx
- HAProxy
- AWS Elastic Load Balancer (ELB)
- Google Cloud Load Balancer
- Azure Load Balancer

☆ SYSTEM DESIGN HANDBOOK ☆

3/10

3. CACHING

What is Caching?

Caching is the process of storing frequently accessed data in a **fast storage** layer so that future requests can be served quickly.

Why do we need Caching?

- Reduces latency and improves performance.
- Reduces load on database and backend.
- Improves scalability.
- Saves bandwidth and resources.

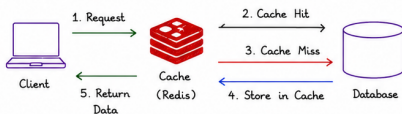
Cache Hit vs Cache Miss

- **Cache Hit** - Data found in cache.
- **Cache Miss** - Data not found in cache, so we fetch from database.

Caching Strategies

1. **Cache Aside (Lazy Loading)**
 - App checks cache.
 - If miss, fetch from DB, store in cache.
 - Most widely used.
2. **Write Through**
 - Write data to cache and DB at the same time.
 - Ensures cache is always up-to-date.
3. **Write Back (Write Behind)**
 - Write data only to cache.
 - Periodically write to DB.
 - Faster, but risk of data loss.
4. **Refresh Ahead**
 - Cache is updated before it expires.
 - Useful for frequently accessed data.

How Caching Works? (Read Flow)



Popular Cache - Redis



- ✓ In-memory key-value store
- ✓ Very fast
- ✓ Supports complex data structures
- ✓ TTL (Time To Live) support
- ✓ Persistence options

Common Data Structures

- String
- Hash
- List
- Set
- Sorted Set
- Bitmap, HyperLogLog, etc.

Cache Eviction Policies

Policy	Description
LRU (Least Recently Used)	Evicts least recently used items.
LFU (Least Frequently Used)	Evicts least frequently used items.
FIFO (First In First Out)	Evicts oldest items first.
TTL (Time To Live)	Evicts items after expiration time.

When Not to Use Cache?

- Data changes very frequently.
- Data is not read frequently.
- Cache is bigger and more expensive than DB.
- When strong consistency is critical.



Interview Tip

Always identify what to cache, how to invalidate, what eviction policy to use and what happens on cache failure.



☆ SYSTEM DESIGN HANDBOOK ☆

4/10

4. DATABASE SCALING

Why do we need Database Scaling?

- Handle growing amount of data.
- Support more users and higher traffic.
- Improve performance and availability.
- Avoid single point of failure.

Scaling Approaches

- Vertical Scaling (Scale Up)**
 - Add more power (CPU, RAM, SSD) to the existing server.
 - Simple but has a hardware limit.
- Horizontal Scaling (Scale Out)**
 - Add more servers to distribute the load.
 - More complex but highly scalable.
- Replication**
 - Copy data to multiple servers.
 - Improves read performance and availability.
- Sharding**
 - Split data across multiple databases.
 - Each shard holds a subset of data.
 - Improves write performance and scalability.
- Partitioning**
 - Split large tables into smaller parts.
 - Helps in managing large datasets.

Replication Topology (Master - Read Replicas)



Database Scaling Strategies

1. Vertical Scaling



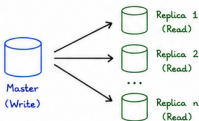
Pros

- Easy to implement
- No changes in application

Cons

- Hardware limit
- Downtime required

2. Horizontal Scaling (Replication)



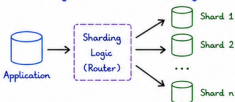
Pros

- High availability
- Better read performance
- Scale out easily

Cons

- Replication lag
- More complex

3. Sharding (Horizontal Partitioning)



Pros

- High write performance
- Handles large data
- Scale out

Cons

- Complex to implement
- Cross-shard joins are difficult

Read Replicas - Use Cases

- Reporting & Analytics
- Search Operations
- Heavy Read Workloads
- Geographically distributed reads

★ Note

Choose the right scaling strategy based on your workload, data size and growth pattern.



Interview Tip

Always discuss the trade-offs of each approach.
Ask questions like: Read heavy or write heavy? Data size?
Consistency required? Budget constraints?



Good system design is all about choosing the right trade-off!

5. CDN (CONTENT DELIVERY NETWORK)

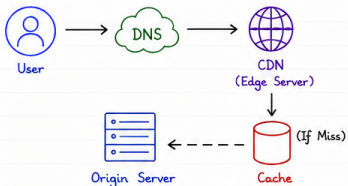
What is CDN?

A CDN is a distributed network of servers (Edge Servers) placed in multiple locations around the world to deliver content to users faster, with lower latency and high availability.

Why do we need CDN?

- Reduces latency by serving content from nearest location.
- Reduces load on origin server.
- Improves website performance.
- Handles more users and traffic.
- Provides DDoS protection.
- Cost effective for serving static content.

How CDN Works? (High Level)



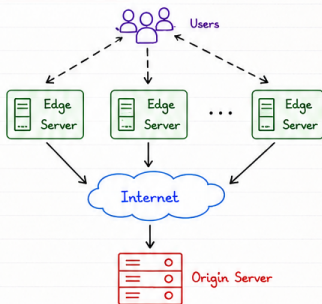
- ① User requests for content (e.g. image, video, file).
- ② DNS resolves to the nearest CDN edge server.
- ③ Edge server checks if content is in cache.
 - If YES (Cache Hit) → Content is served to user.
 - If NO (Cache Miss) → Content is fetched from origin server, cached and then served.
- ④ Subsequent requests are served from cache.

What is Cached?

- Static content → Images, CSS, JS, Fonts, Videos
- API responses (sometimes)
- HTML pages (using edge caching)
- Large files, downloads



CDN Architecture



Key Terms

- Edge Server - Servers close to users.
- Origin Server - Main server where original content is stored.
- Cache - Temporary storage at edge.
- TTL (Time To Live) - Cache expiration time.
- Purge / Invalidate - Remove content from cache before it expires.

Types of CDN Content

- Static Content - Images, videos, CSS, JS, Fonts
- Dynamic Content - API responses, HTML pages
- Streaming Content - Live / On-demand videos

Popular CDNs

- Cloudflare
- Amazon CloudFront
- Akamai
- Fastly
- Microsoft Azure CDN
- Google Cloud CDN

Best Practices

- ✓ Set proper cache headers and TTL.
- ✓ Compress content (Gzip / Brotli).
- ✓ Use versioned files for cache busting.
- ✓ Use HTTPS everywhere.
- ✓ Monitor cache hit ratio.



Interview Tip

Always identify what content can be cached.
CDN is not for dynamic, user-specific or highly sensitive data.



CDN makes your system faster, scalable and user-friendly!

6. MESSAGE QUEUES

What is a Message Queue?

A Message Queue is a communication channel that enables services (or applications) to communicate **asynchronously** by sending messages.

Why do we need Message Queues?

- Decouple services and components.
- Handle high traffic spikes.
- Improve system scalability.
- Ensure reliability with retries.
- Enable asynchronous processing.
- Buffer messages when consumers are down.

How Does It Work?



- ① Producer sends message to the queue.
- ② Message is stored in the queue.
- ③ Consumer reads the message and processes it.
- ④ Message is removed from the queue after successful processing.

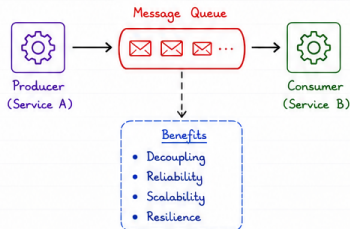
Key Features

- Asynchronous Communication
- Durability (messages are stored until consumed)
- Reliability (acknowledgements, retries)
- Scalability
- Ordering (FIFO or custom)
- Load Balancing
- Fault Tolerance

Important Terms

- Producer** - The service that sends the message.
- Consumer** - The service that consumes the message.
- Queue** - The buffer that holds messages.
- Message** - The data sent between producer and consumer.
- Ack** - Acknowledgement that message was received.
- DLQ** - Dead Letter Queue (for failed messages).

Message Queue Flow



Popular Message Queue Systems

System	Highlights
RabbitMQ	AMQP Protocol, supports multiple languages, reliable, flexible routing.
Apache Kafka	High throughput, distributed, persistent logs, pub/sub, stream processing.
Amazon SQS	Fully managed, highly available, scalable, integrated with AWS.
Azure Service Bus	Fully managed, supports queues, topics, subscriptions.
Google Pub/Sub	Global, scalable, real-time messaging service by GCP.

Message Delivery Guarantees

- **At Most Once** - Message might be lost.
- **At Least Once** - Message will be delivered at least once (duplicates possible).
- **Exactly Once** - Message will be delivered once and only once (most strict).

Common Use Cases

- Background Jobs (e.g., email sending)
- Order Processing
- Payment Processing
- Log Aggregation
- Notification System
- Event Driven Architecture

Handling Failures

- ✓ Use Retries with Exponential Backoff.
- ✓ Move failed messages to DLQ.
- ✓ Ensure Idempotent Consumers.
- ✓ Monitor queue length and consumer lag.

Interview Tip

Always clarify in interviews:

- > Why do we need a message queue here?
- > What are the message delivery guarantees?
- > How will we handle failures?
- > How do we ensure idempotency?

Good system design is all about decoupling, scalability & reliability!

7. MICROSERVICES

What are Microservices?

An architectural style where an application is built as a collection of **small, independent services** that communicate over a network.

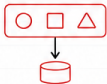
Why Microservices?

- Independent deployment of services.
- Scalability - Scale only what you need.
- Flexibility - Use different tech stacks.
- Fault isolation - Failure in one service doesn't bring down the whole system.
- Faster development by small teams.

Monolith vs Microservices

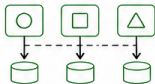
Monolith

- Single codebase
- One deployable
- Tight coupling
- Hard to scale
- Tech stack is fixed

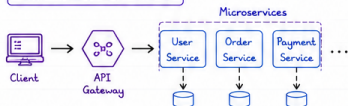


Microservices

- Multiple codebases
- Multiple deployables
- Loose coupling
- Easy to scale
- Polyglot (any tech)



How Microservices Work?



- 1 Client sends request to API Gateway.
- 2 Gateway routes request to the appropriate service.
- 3 Service processes the request and may call other services.
- 4 Each service has its own database.
- 5 Response flows back to the client.

Communication Between Services

- Synchronous - REST APIs, gRPC
- Asynchronous - Message Queues, Events

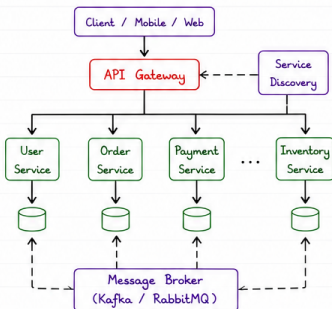


Database per Service

Each microservice owns its data.
No shared database.



Microservices Architecture



Key Components

- **API Gateway** - Single entry point for clients.
- **Service Discovery** - Helps services find & talk to each other (e.g., Eureka, Consul).
- **Load Balancer** - Distributes traffic to service instances.
- **Config Server** - Centralized configuration for all services.
- **Message Broker** - Enables async communication and event streaming.
- **Monitoring & Logging** - Track health, logs, metrics (Prometheus, ELK, Grafana).

Challenges

- Operational Complexity
- Distributed System Complexity
- Network Latency
- Data Consistency
- Testing is Harder



Best Practices

- ✓ Keep services small and focused.
- ✓ Design around business capabilities.
- ✓ Use async communication where possible.
- ✓ Implement circuit breaker and retries.
- ✓ Centralize logging, monitoring and alerts.



Interview Tip

Always discuss trade-offs. Explain why microservices are a good fit for the use case, and how you will deal with complexity.



Microservices are not a silver bullet. Use them when the system is large, complex and needs to scale independently.

8. LOAD BALANCER

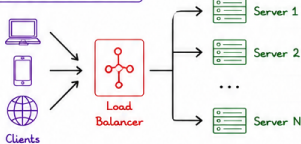
What is Load Balancer?

A Load Balancer distributes incoming traffic across multiple servers to ensure no single server is overwhelmed, maximizing throughput, minimizing latency and improving availability.

Why do we need Load Balancer?

- Distribute traffic evenly across servers.
- Prevent any single server from becoming a bottleneck.
- Improve availability and fault tolerance.
- Enable horizontal scaling.
- Perform health checks.
- SSL termination and other offloading.

How Does It Work?



- ① Client sends request to the Load Balancer.
- ② Load Balancer selects a server based on the balancing algorithm.
- ③ Request is forwarded to the selected server.
- ④ Server processes the request and sends the response back through the Load Balancer.
- ⑤ Load Balancer returns the response to the client.

Load Balancing Algorithms

Algorithm	Description
Round Robin	Requests are distributed sequentially across servers.
Least Connections	Requests are sent to the server with the least active connections.
IP Hash	Based on client IP, same IP always goes to the same server.
Least Response Time	Requests are sent to the server with the lowest response time.
Random	Requests are sent to a random server.

Types of Load Balancer

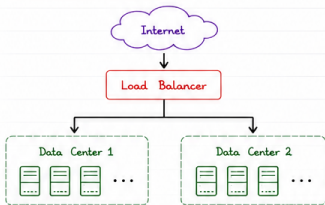
- **L4 (Transport Layer)** - Works with IP and port (TCP/UDP). Faster, simple and efficient.
- **L7 (Application Layer)** - Works with HTTP/HTTPS. Can route based on content (URL, headers, etc.).



Interview Tip

Always consider - Where should the load balancer be placed?
How will it perform health checks?
What if a server or the load balancer fails?

Load Balancer Placement



Popular Load Balancers

Hardware Based

- F5 BIG-IP
- Citrix NetScaler
- A10 Networks

Software / Cloud Based

- NGINX
- HAProxy
- AWS ALB / NLB
- Google Cloud LB

Health Check

- Load balancer periodically checks the health of servers.
- If a server is unhealthy, it stops sending traffic to it.
- Health check methods: HTTP, HTTPS, TCP, Ping, etc.

Handling Failures



Key Considerations

- **Scalability** - Can the LB handle increased traffic?
- **High Availability** - LB itself should not be a single point of failure.
- **Stickiness (Session Affinity)** - Keep a user on the same server when needed.
- **SSL Termination** - Offload SSL to reduce server load.
- **Monitoring & Metrics** - Monitor traffic, latency, errors, and health.



Use Cases

- Web applications
- APIs and microservices
- Databases (Read replicas)
- Streaming services
- Online gaming
- E-commerce platforms



Good load balancing is key to a scalable, reliable and high-performing system!



9. CACHING STRATEGIES

What is Caching?

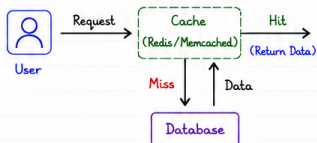
Caching is storing frequently accessed data in a **fast storage** (like memory) so that future requests can be served **quickly**.

Why do we need Caching?

- Reduces latency.
- Reduces load on database and servers.
- Improves throughput and performance.
- Saves bandwidth and cost.
- Improves user experience.



How Does Cache Work?



- ① User sends a request for data.
- ② System checks cache.
 - If **Hit** → return data from cache.
 - If **Miss** → fetch from DB, return to user and store in cache.
- ③ Next time, data is served from cache (faster).

Common Caching Patterns



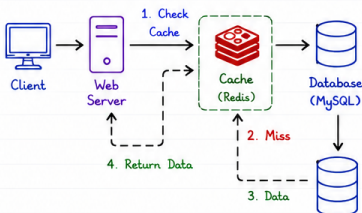
Pattern	Description
Cache-Aside (Lazy Loading)	Application checks cache first, on miss → load from DB and cache it.
Write-Through	Write to cache and DB simultaneously.
Write-Behind (Write-Back)	Write to cache first, then async write to DB.
Refresh-Ahead	Cache is updated before it expires.
Cache Invalidation (Expire)	Remove or refresh stale data from cache.

Cache Eviction Policies

- LRU (Least Recently Used)
- LFU (Least Frequently Used)
- FIFO (First In First Out)
- TTL (Time To Live)



Caching Architecture (Example)



Popular Caching Systems

Redis	In-memory data store, supports rich data structures.
Memcached	High performance, simple key-value store.
Ehcache	Java based, supports multiple cache layers.
Amazon ElastiCache	Managed Redis/Memcached service on AWS.

Best Practices

- ✓ Cache only read-heavy and frequently used data.
- ✓ Use appropriate TTL for data.
- ✓ Implement cache invalidation properly.
- ✓ Avoid caching sensitive or user-specific data.
- ✓ Monitor cache hit ratio.
- ✓ Don't cache everything! (Know your data)



Challenges

- Cache Invalidation (stale data)
- Choosing the right TTL
- Cache Penetration (invalid keys)
- Cache Breakdown (hot key issue)
- Memory Limitations



When Not to Use Cache?

- ✗ Data changes very frequently.
- ✗ Data is not read often.
- ✗ Real-time consistency is critical.



Interview Tip

- ☆ Explain when and why you will use caching.
- ☆ Discuss trade-offs, eviction policies and consistency.
- ☆ Don't forget to talk about cache invalidation!



Good caching
makes your system
10x faster!



10. NOTIFICATION SYSTEM

What is a Notification System?

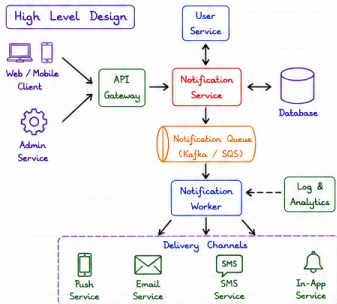
A Notification System delivers messages to users through various channels like push notifications, email, SMS, in-app notifications, etc.

It should be reliable, fast and highly available.

Requirements

- Send notifications to millions of users.
- Support multiple channels (Push, Email, SMS, In-App).
- Personalized and scheduled notifications.
- High throughput and low latency.
- Retry mechanism for failed notifications.
- Track delivery status (Sent, Delivered, Failed).

High Level Design



Flow

- 1 Client or Admin creates a notification.
- 2 API Gateway forwards request to Notification Service.
- 3 Notification Service validates and stores in DB.
- 4 It publishes the notification to a Message Queue.
- 5 Workers consume from the queue and send via appropriate channel.
- 6 Delivery status is updated and logged.

Data Model (Notification)

- | | |
|---|---|
| <ul style="list-style-type: none"> • id (UUID) • user_id • title • message • channel (push/email/sms/in-app) | <ul style="list-style-type: none"> • data (JSON) • scheduled_at • status (pending/sent/delivered/failed) • created_at • updated_at |
|---|---|

Scalability Considerations

- Use message queues for decoupling and scaling.
- Workers can be scaled horizontally.
- Use rate limiting for third-party services (SMS/Email).
- Store templates and prefer pre-rendered content.
- Use caching for user preferences and tokens.



Delivery Channels

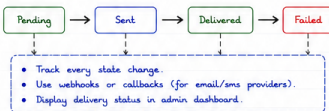
Channel	Use Case	Pros	Cons
Push	App notifications (Android / iOS)	Real-time, High engagement	Needs app installed
Email	Marketing, alerts, receipts	Reliable, Support rich content	Can go to spam
SMS	OTP, alerts, critical messages	High open rate	Expensive, Limited length
In-App	In-app messages, updates	Instant, No external fees	Limited to active users

Retry & Failure Handling

- Retry failed notifications with exponential backoff.
- Use Dead Letter Queue (DLQ) for max retry failures.
- Mark status as failed and alert if needed.
- Store failure reasons for analytics.



Notification States



Key Components

- API Gateway - Rate limiting, auth, routing.
- Notification Service - Validates and enqueues.
- Message Queue - Decouples and buffers.
- Worker - Processes and sends notifications.
- Channel Services - Integrates with providers.
- Database - Stores notifications and status.
- Log & Analytics - Monitoring and reporting.



Best Practices

- ✓ Batch notifications where possible.
- ✓ Use templates and placeholders.
- ✓ Don't block user request - respond quickly.
- ✓ Monitor queue lag, failure rate, success rate.
- ✓ Set up alerts for high failure or delay.
- ✓ Provide user preferences (opt-in/opt-out).



Common Use Cases

- | | |
|--|---|
| <ul style="list-style-type: none"> • Password reset • Order confirmation • Shipping updates • Payment alerts | <ul style="list-style-type: none"> • Promotions & offers • Social activity updates • System alerts • Reminder notifications |
|--|---|



Metrics to Monitor

- Notifications sent per second
- Delivery success rate
- Failure rate
- Queue length / lag
- Channel wise delivery state
- Latency (end-to-end)



Interview Tip

- When asked to design a Notification System:
- Start with requirements and assumptions.
 - Discuss high level design and data flow.
 - Explain components, scaling, and failure handling.
 - Talk about monitoring and metrics.



A well-designed notification system keeps users informed and engaged without being intrusive!